拷贝艺术——画布状态栈

1）画布的绘制模式

在理解画布状态栈之前，我们先来加深对 `canvas` 本身的认识：

`canvas(画布)` 见名知意，就是一个可以让我们开发者在上面“画画”（编程作画）的工具，最终结果就是呈现一幅图像。

在 `DOM` 中我们用 `<div></div>` 创建一个方块的时候，浏览器自动创建一个页面模型（渲染树），里面会记录并维护这个方块的信息，最后浏览器将模型转化为绘制命令去**帮我们**绘制到屏幕上，后续有修改的时候就更新模型重新生成指令绘制。但是画布就不一样了，它只是个“画画”的地方，它才不会去记录这幅画中是有个矩形还是有个圆形，呈现什么图形完全取决我们自己调了什么绘制指令。

简单来说就是画布是亲自上阵调指令绘制的，而 `DOM` 是我们给了个配置让浏览器帮忙调指令绘制（了解更多区别：DOM 与画布）。

当然两种绘制模式各有各的优劣，不过这里并不展开这个，这里只是想说明：**当你画下一个红色矩形的时候，它才不会帮你记住上一个画的是黄色圆形。**

所以如果要记住要怎么做呢？那就需要一个能帮我们记住这些状态的工具：状态栈。

这里也说明了一件事情，状态栈相对而已只是个辅助工具，而不是必需品，我们完全可以自己记录状态。

这个工具提供了两个API：

`save()` 保存画布状态

`restore()` 恢复画布状态

接下来我们从 `blink::canvas` 的源码来认识它们。

2） `save()`

> CanvasRenderingContext2D.save() 是 Canvas 2D API 通过将当前状态放入栈中，保存 canvas 全部状态的方法。—— MDN

如果只是理解表面的意思，对于使用者来说其实已经足够了，但是他将不会意识到这个操作所需要付出的代价。

请思考以下问题：

① 放入栈的最新状态应该来自哪里？

② 只有被修改的状态才会放入栈中吗？

在我没看源码前我觉得应该是遍历一遍context2d，然后对比并存储了所有修改后的新状态，原因是可以减少内存使用。但实际上完全不是，来，一起看看源码：

```
// 存储在堆内存中
HeapVector<Member<CanvasRenderingContext2DState>> state_stack_;

ALWAYS_INLINE CanvasRenderingContext2DState& GetState() const {
    // 这里的back用于获取数组最后一个值
    return *state_stack_.back();
}

void BaseRenderingContext2D::save() {
    // 这里的push_back“近似”于JavaScript数组中的push方法
    state_stack_.push_back(GetState());
}
```

换成JavaScript写法，类似：

```
function save () {
  stack.push(stack[stack.length - 1]);
}
```

“就是将栈顶元素再加入栈？不应该吧，这个数据不是引用类型的吗？”

那是因为你把这里的 `push` 完成看成了 `JavaScript` 的 `push` 了。

如果说C++的 `push_back()` 是一次深拷贝的操作，你是不是就豁然开朗了呢。

一句话概括， `save()` 在源码的实现中就是将状态栈的最后一项（最新的状态），拷贝一份后再加入栈中，用这种方式来实现状态的保存。

很巧妙吧，但这也暴露了一些问题，那就是深拷贝的性能以及内存浪费的问题。并且我在源码中也没有看到类似比较的操作，也就是它完全不关心状态有没有更改，只要调用 `save()` 就帮你拷贝一整份。

3） `restore()`

CanvasRenderingContext2D.restore() 是 Canvas 2D API 通过在绘图状态栈中弹出顶端的状态，将 canvas 恢复到最近的保存状态的方法。如果没有保存状态，此方法不做任何改变。—— MDN

恢复状态的API就没啥特别之处了，如MDN所写弹出栈顶状态，恢复成新的栈顶状态，不过这里有一点值得一提，请看源码：

```
void BaseRenderingContext2D::restore() {
  if (state_stack_.size() <= 1)
    return;
  PopAndRestore();
```

```
}
```

这里当栈元素数量等于1时就返回了，为什么不等于0呢？

其实也就是因为 `save()` 的实现，它必须要有一个默认状态作为拷贝对象，所以栈的第一个元素必然是画布的默认状态。

另外这也说明一件事，如果在修改默认状态前没有通过 `save()` 拷贝一份，那将导致默认状态被修改而且无法通过 `restore` 恢复，直到你自己手动改回默认值。

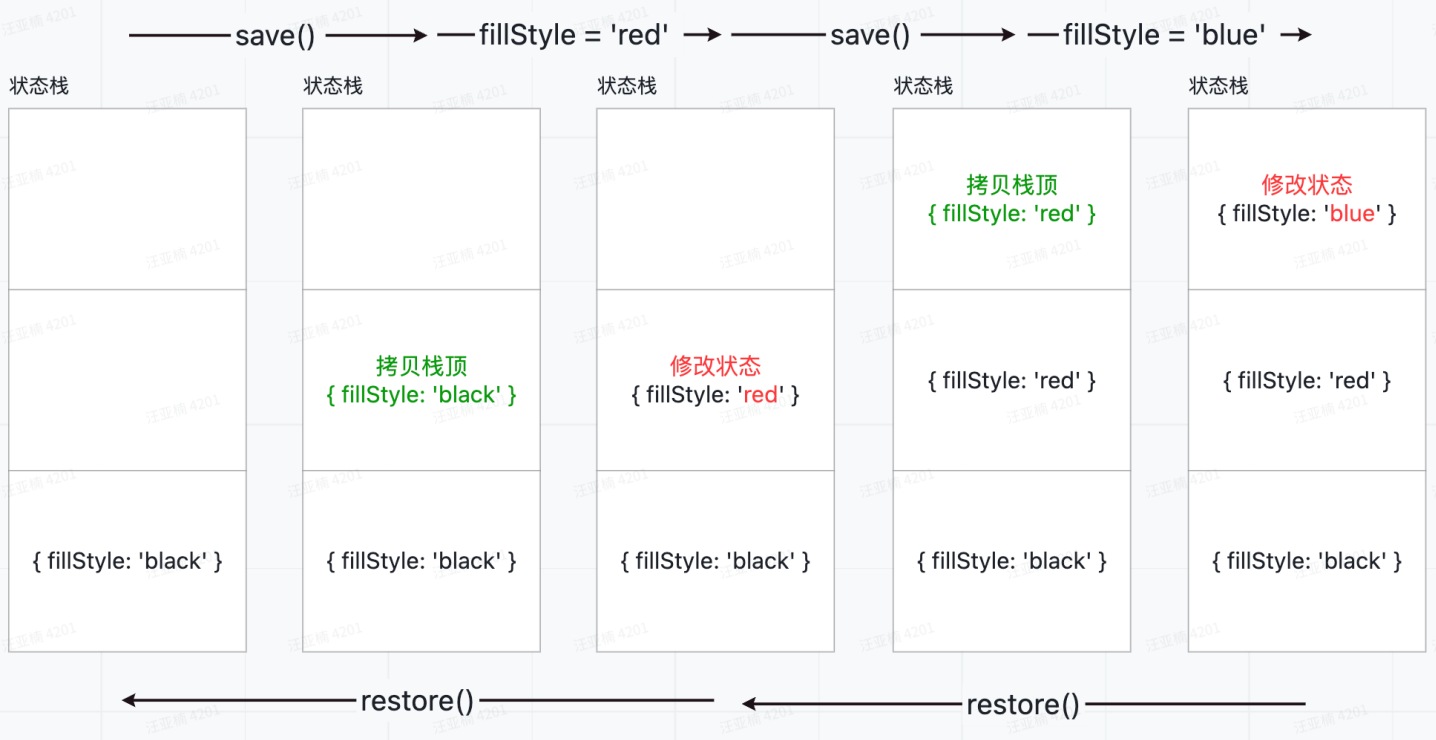
`restore()` 相比 `save()` 而言虽然不需要深拷贝等操作，但也好不到哪去，它需要执行恢复画布状态的操作，这里的状态可不止是填充颜色那些，还包括画布变换那些，所以也是很耗费性能的。

4）源码原理图演示

https://codepen.io/huanjinliu/pen/JjevxvZ

```
// 上面的代码拆去循环类似于以下操作
const canvas = document.getElementById('canvas');
const ctx = canvas.getContext('2d');

// 保存状态（拷贝）
ctx.save();
// 改栈顶状态
ctx.fillStyle = 'red';
// 保存状态（拷贝）
ctx.save();
// 改栈顶状态
ctx.fillStyle = 'blue';
// 恢复状态（弹出）
ctx.restore();
// 恢复状态（弹出）
ctx.restore();
```



5）性能测试

① 单纯 `save()` 测试100w次调用：多数200ms+

```
const canvas = document.getElementById('canvas');
const ctx = canvas.getContext('2d');

console.time('performation');
for (let i = 0; i < 1000000; i++) ctx.save();
console.timeEnd('performation');
```

② 单纯 `restore()` 测试100w次调用：多数150ms+

```
const canvas = document.getElementById('canvas');
const ctx = canvas.getContext('2d');

// 这里必须成对save()的调用，不然restore()没有效果
for (let i = 0; i < 1000000; i++) ctx.save();
console.time('performation');
for (let i = 0; i < 1000000; i++) ctx.restore();
console.timeEnd('performation');
```

③ `save()` 和 `restore()` 同时使用测试100w次调用：多数180ms+

```
const canvas = document.getElementById('canvas');
```

```
const ctx = canvas.getContext('2d');

console.time('performation');
for (let i = 0; i < 1000000; i++) {
  ctx.save();
  ctx.restore();
}
console.timeEnd('performation');
```

这里比单独使用耗时少，猜测是C++内存分配的原因，C++的数组是要动态分配内存的，这里一保存就恢复，数组完全不需要像①②那样需要不断扩展数组。

> “已知元素是连续存储的，当我们在容器内添加一个元素时，如果容器中已经没有空间容纳新的元素，此时，由于元素必须连续存储以便索引访问，所以不能在内存中随便找个地方存储这个新元素，于是，vector必须重新分配存储空间，用来存放原来的元素以及新添加的元素，存放在旧存储空间中的元素被复制到新存储空间里，接着插入新元素，最后撤销旧的存储空间。”
>
> “为了使vector容器实现快速的内存分配，其实际分配的容量要比当前所需的空间多一些。”
>
> C++语言之vector内存分配技术

6）可能的滥用情况

① 不合理的调用时机

永远要留意循环中的 `save&restore` ，如果循环中的状态是中间更改，直接在循环外进行一次保存和恢复操作即可，无需在每次循环内都去保存和恢复，这在做无用功。

举个例子：生成10w个随机颜色随机位置的1px方块。

```
const canvas = document.getElementById('canvas');
const ctx = canvas.getContext('2d');
// GOOD
+ ctx.save();
for (let i = 0; i < 100000; i++) {
  // BAD
  - ctx.save();
  ctx.fillStyle =  rgb(
    ${Math.floor(Math.random() * 255)},
    ${Math.floor(Math.random() * 255)},
    ${Math.floor(Math.random() * 255)}
  );
  ctx.fillRect(Math.random() * canvas.width, Math.random() * canvas.height, 1, 1);
  // BAD
  - ctx.restore();
}
```

```
// GOOD
+ ctx.restore();
```

```
// 滥用的情况
save();
ctx.fillStyle = 'red';
restore();

save();
ctx.fillStyle = 'green';
restore();

save();
ctx.fillStyle = 'blue';
restore();

...
```

```
// 移除不必要的调用
save();
ctx.fillStyle = 'red';
- restore();

- save();
ctx.fillStyle = 'green';
- restore();

- save();
ctx.fillStyle = 'blue';
restore();

...
```

> 注意：务必留意循环中的使用，个人感觉这是最容易造成滥用的地方。

② 毫无意义的调用

`save()` 并不是什么画布信息都能记录，比如就不能记录路径的信息，所以在仅路径操作中使用是毫无意义的。

举个例子：

```
const canvas = document.getElementById('canvas');
const ctx = canvas.getContext('2d');
// BAD
- ctx.save();
ctx.beginPath();
ctx.moveTo(50, 50);
ctx.lineTo(100, 100);
// BAD
- ctx.restore();
```

变换信息、裁剪信息、样式、全局画布配置等状态是可以记录的，具体看这个：
CanvasRenderingContext2D: save() method - Web APIs | MDN

考考你们：

① `save()` 和 `restore()` 可以用来做“时光机”吗？——画布的撤回重做。

解答：保存状态 ≠ 保存绘制指令，想拿这个做画面的撤回重做是完全行不通的，

没用 `save` 时：

“当你画下一个红色矩形的时候，它才不会帮你记住上一个画的是黄色圆形。”

用了 `save` 时：

“当你画下一个红色矩形的时候，它帮你记住上一次绘制用的是黄色，但是它却不知道你画的是圆形。”

② 以下程序会造成什么问题：

```
while(1) {
  if (condition) break;
  ctx.save();
  ctx.fillStyle = 随机颜色值;
  ctx.restore();
  ctx.save();
}
```

解答：

1. 首先就是错误的调用时机导致性能变差。
2. 其次，`save` 和 `restore` 肯定是成对使用的，如果在某段循环中调的 `save` 方法调用次数多于 `restore`，你要注意了，你的程序总有爆内存的时刻，因为状态栈在无限拷贝栈顶状态。